

LABORATORIO 3.5

Control remoto con sockets y joystick

Preparado por:
Luisdavid Colina V24766381
Samantha Ramírez V31307714

1. Desarrollo de actividades

1.1. Actividad 6.1: Servidor de control remoto (socket server)

Enunciado: Implemente un código de Processing que actúe como servidor TCP. El servidor debe recibir comandos de ángulos enviados desde un cliente remoto y retransmitirlos al servo físico. Se debe validar el rango del valor recibido antes de aplicarlo al hardware.

Processing incluye la librería nativa `processing.net` que permite crear servidores y clientes TCP sin dependencias externas [2]. En el código se creó un objeto `Server` escuchando en el puerto 12345 (Código 1). El servo se configura igual que en la actividad anterior: pin 3 en modo `Arduino.SERVO` con posición inicial de 90°.

Código 1: Inicialización del servidor TCP y del servo en `setup()`.

```
import processing.serial.*;
import processing.net.*;
import cc.arduino.*;

Arduino arduino;
Server miServidor;
int servoPin = 3;
int anguloServo = 90;

arduino = new Arduino(this, Arduino.list()[2], 57600);
arduino.pinMode(servoPin, Arduino.SERVO);
arduino.servoWrite(servoPin, anguloServo);

miServidor = new Server(this, 12345);
```

En `draw()`, el servidor consulta si hay algún cliente con datos disponibles mediante `miServidor.available()`. Si hay datos, se lee la cadena hasta el carácter delimitador `'\n'` que el cliente agrega al final de cada comando. El uso de ese delimitador es importante porque TCP es un protocolo de flujo de bytes sin bordes de mensaje: sin él no habría forma confiable de saber dónde termina un valor y dónde empieza el siguiente (Código 2).

Código 2: Recepción, validación y aplicación del ángulo en `draw()`.

```
Client clienteEntrante = miServidor.available();

if (clienteEntrante != null) {
    String cadena = clienteEntrante.readStringUntil('\n');

    if (cadena != null) {
        cadena = trim(cadena);
        try {
            int nuevoAngulo = int(cadena);
            if (nuevoAngulo >= 0 && nuevoAngulo <= 180) {
                anguloServo = nuevoAngulo;
                arduino.servoWrite(servoPin, anguloServo);
            }
        } catch (Exception e) {
            println("Error de trama: " + e.getMessage());
        }
    }
}
```

La validación del rango `[0, 180]` antes de llamar a `servoWrite()` es una protección de hardware: valores fuera de ese rango pueden dañar mecánicamente el servo o generar una señal PWM incorrecta. La interfaz visual del servidor muestra en pantalla el último ángulo recibido, lo que facilita la depuración sin necesidad de abrir la consola.

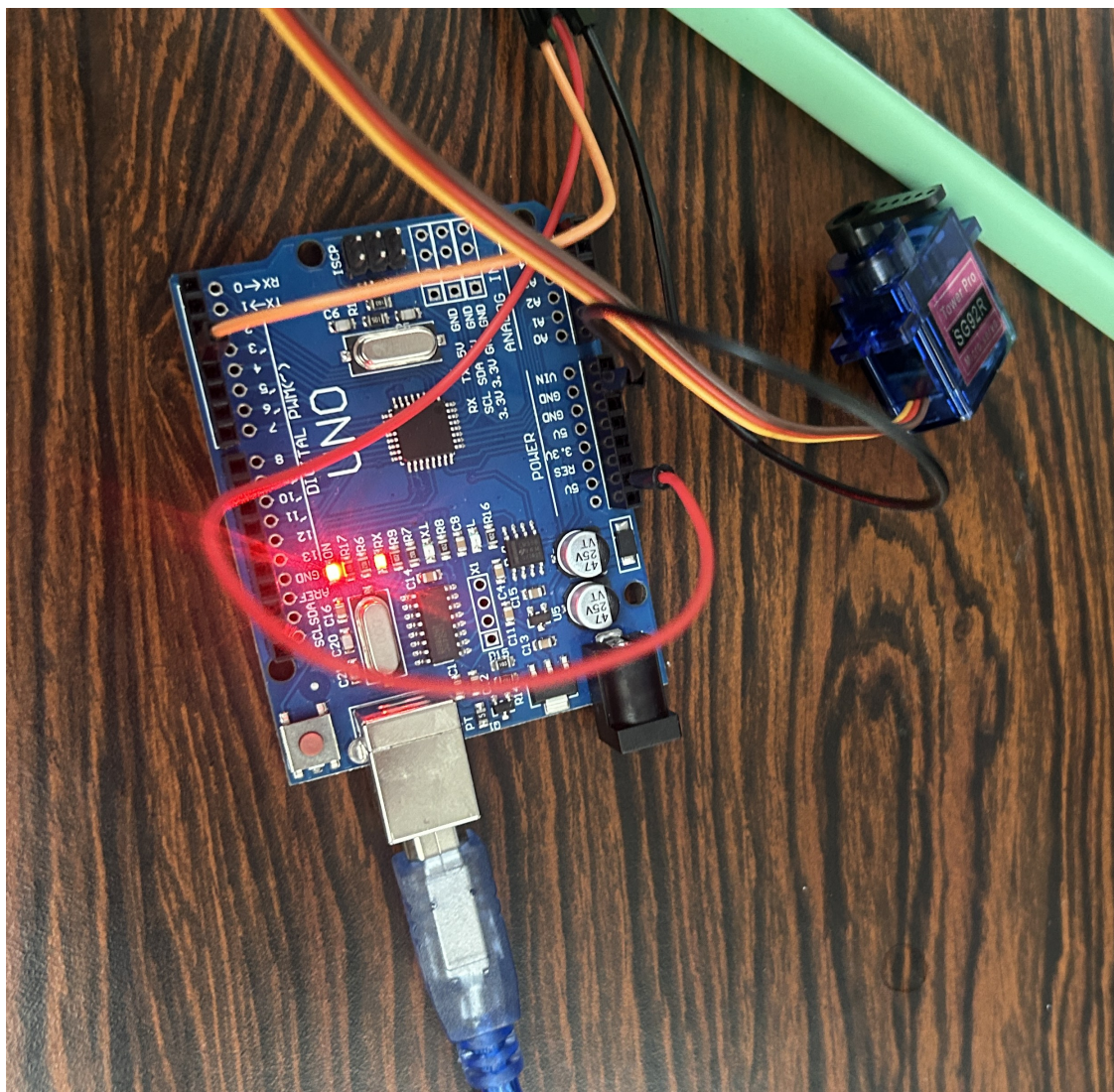


Figura 1: IMG

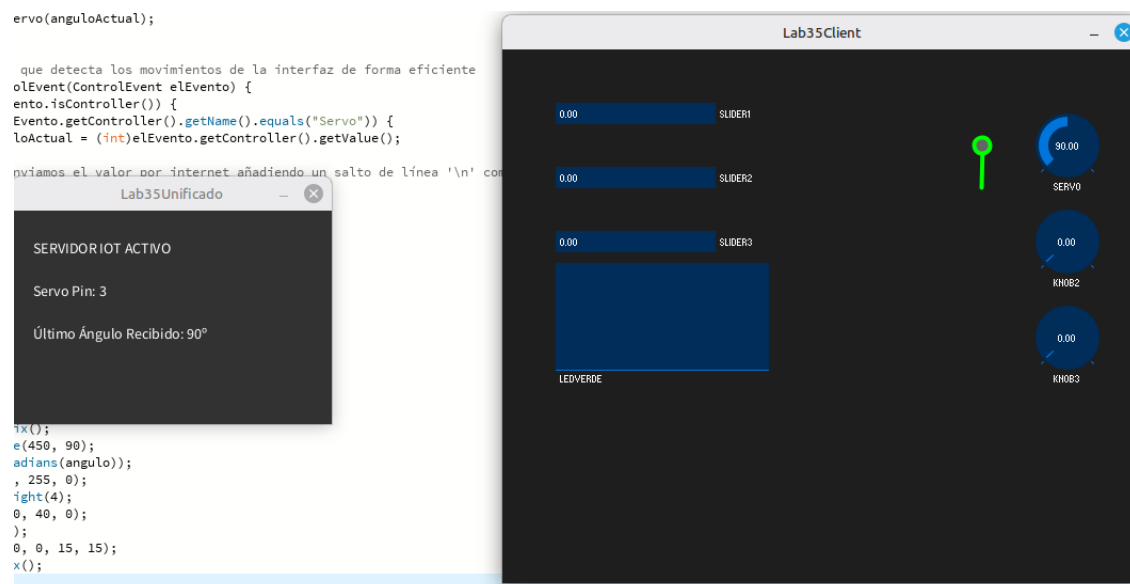


Figura 2: IMG

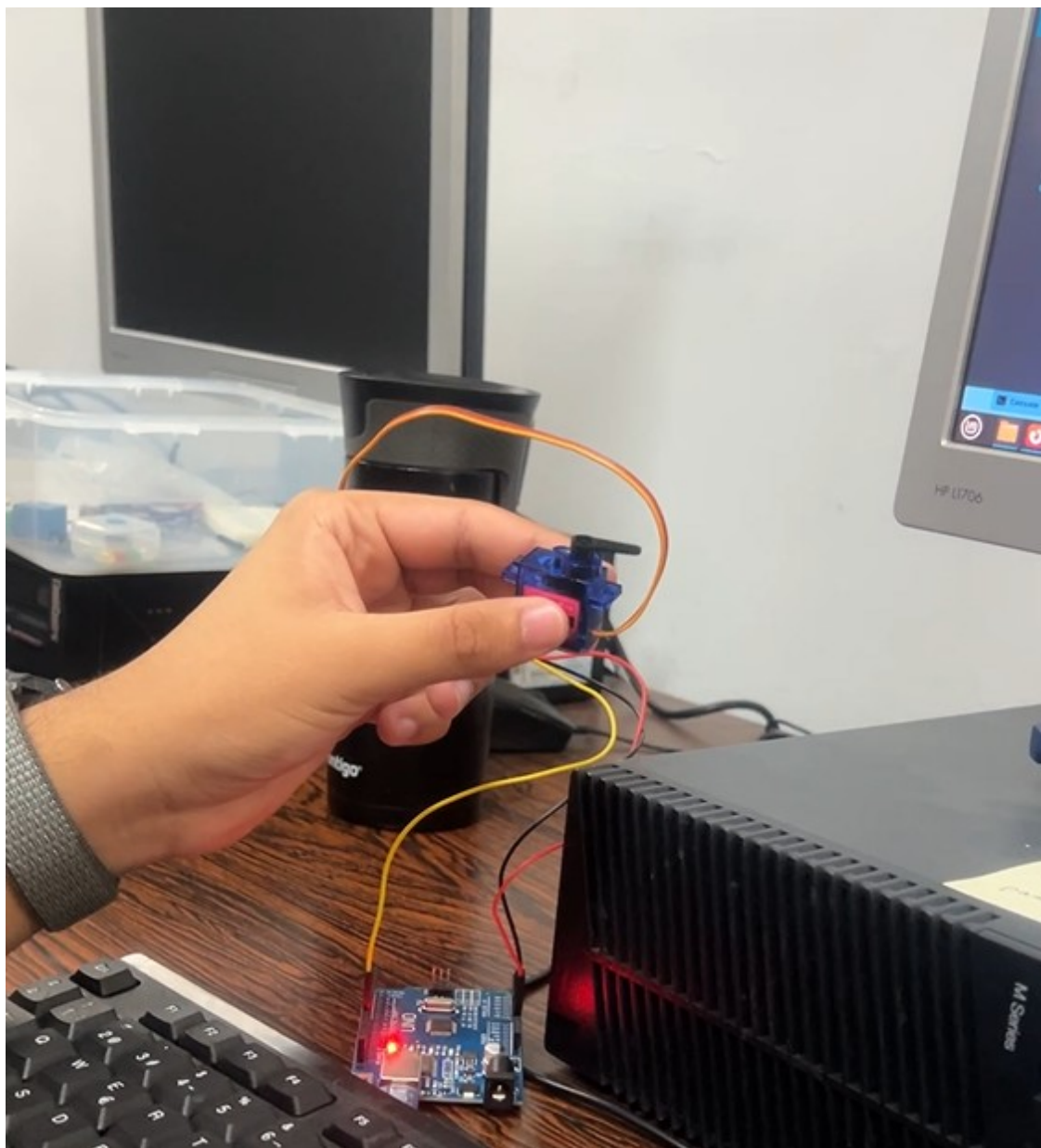


Figura 3: IMG

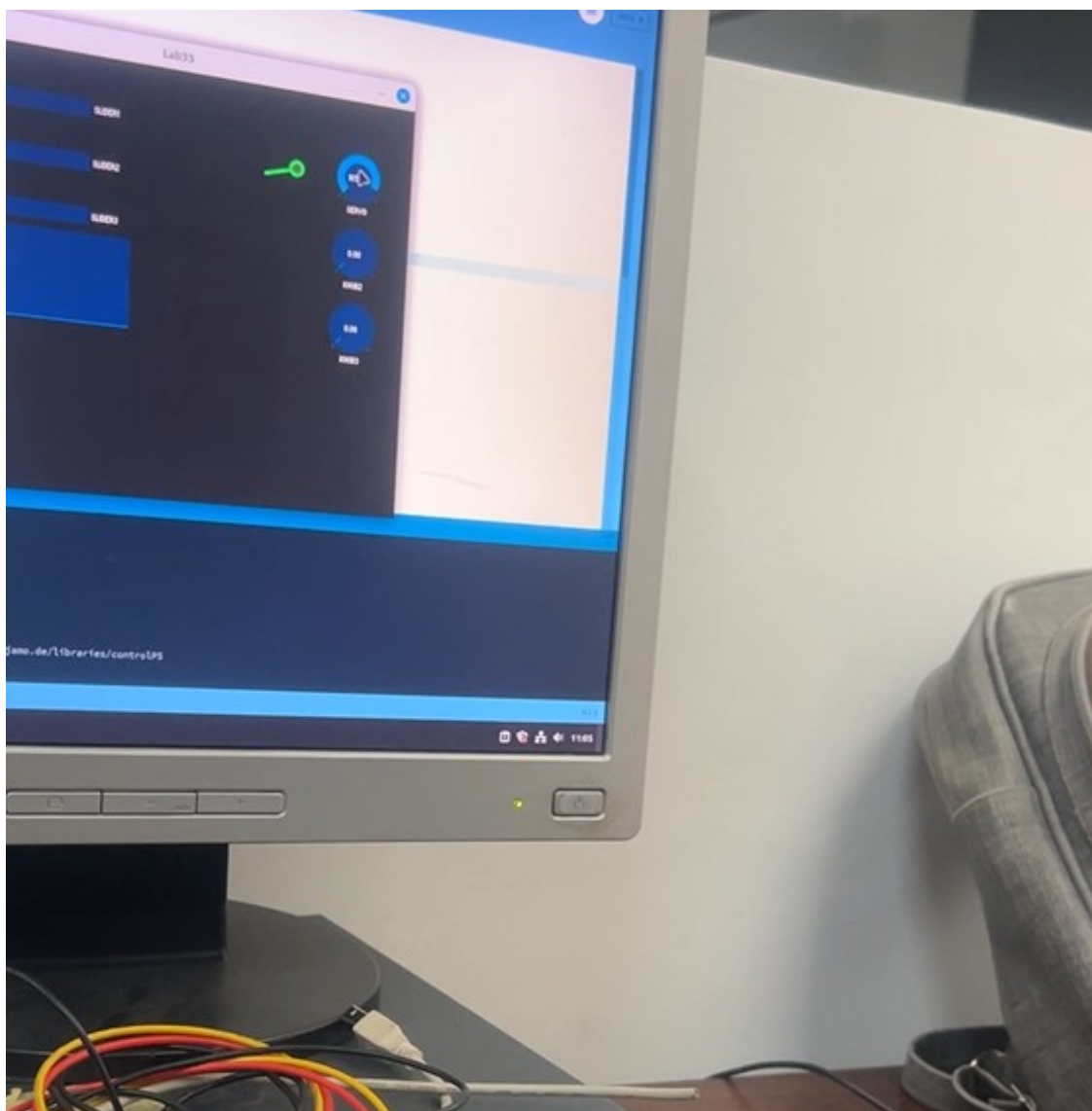


Figura 4: IMG

1.2. Actividad 6.2: Cliente remoto con interfaz gráfica (socket client)

Enunciado: Desarrolle un cliente TCP en Processing que permita controlar remotamente el servo del servidor. La interfaz debe incluir sliders, knobs y una representación gráfica del servo. Solo el knob de servo debe transmitir datos al servidor; los sliders operan de forma puramente visual.

El cliente importa `controlP5` para los controles de interfaz y `processing.net` para la conexión TCP. Al inicio, se crea un objeto `Client` apuntando a la dirección IP del equipo servidor y al puerto expuesto a través del reenvío de puertos de la red local (Código 3). Así, el cliente conecta al puerto 8080 de la IP pública mientras el servidor escucha en el puerto 12345 localmente.

Código 3: Creación del cliente TCP y los controles en `setup()`.

```
import controlP5.*;
import processing.net.*;

Client miCliente;
ControlP5 cp5;
int anguloActual = 0;

// IP del equipo con el Arduino. Puerto: 8080 (redirigido al 12345 del servidor)
miCliente = new Client(this, "190.169.74.67", 8080);

cp5.addSlider("slider1").setPosition(50, 50).setRange(0, 255).setSize(150, 20);
cp5.addSlider("slider2").setPosition(50, 110).setRange(0, 255).setSize(150, 20);
cp5.addSlider("slider3").setPosition(50, 170).setRange(0, 255).setSize(150, 20);

cp5.addKnob("Servo").setPosition(500, 60).setRadius(30).setRange(0, 180).setValue(90);
cp5.addKnob("knob2").setPosition(500, 150).setRadius(30).setRange(0, 255);
cp5.addKnob("knob3").setPosition(500, 240).setRadius(30).setRange(0, 255);
```

La transmisión al servidor se produce únicamente a través del callback `controlEvent()`, que se invoca automáticamente por `ControlP5` cada vez que el usuario mueve un control. Solo el knob llamado "Servo" genera tráfico de red; el resto de los controles son exclusivamente visuales en esta implementación (Código 4).

Código 4: Envío del ángulo al servidor en `controlEvent()`.

```
void controlEvent(ControlEvent elEvento) {  
    if (elEvento.isController()) {  
        if (elEvento.getController().getName().equals("Servo")) {  
            anguloActual = (int)elEvento.getController().getValue();  
            if (miCliente.active()) {  
                miCliente.write(anguloActual + "\n");  
            }  
        }  
    }  
}
```

El valor se envía como cadena de texto seguida del carácter `'\n'`, que es el mismo delimitador que el servidor espera. La comprobación de `miCliente.active()` evita que el programa lance una excepción si la conexión se interrumpe.

En `draw()` se renderizan los LEDs virtuales mediante la función `dibujarLED()`, que usa el canal alfa del color para simular el brillo relativo de cada canal RGB, y se llama a `dibujarServo()` para animar el brazo en pantalla con el ángulo actual (Código 5).

Código 5: Renderizado de LEDs virtuales y brazo del servo en `draw()`.

```
void draw() {
    background(30);
    int val1 = (int)cp5.getController("slider1").getValue();
    int val2 = (int)cp5.getController("slider2").getValue();
    int val3 = (int)cp5.getController("slider3").getValue();

    dibujarLED(260, 60, color(255, 0, 0), val1);
    dibujarLED(260, 120, color(0, 255, 0), val2);
    dibujarLED(260, 180, color(0, 0, 255), val3);
    dibujarServo(anguloActual);
}

void dibujarLED(int x, int y, color c, int val) {
    fill(red(c), green(c), blue(c), val);
    noStroke();
    ellipse(x, y, 40, 40);
}

void dibujarServo(int angulo) {
    pushMatrix();
    translate(450, 90);
    rotate(radians(angulo));
    stroke(0, 255, 0);
    strokeWeight(4);
    line(0, 0, 40, 0);
    fill(100);
    ellipse(0, 0, 15, 15);
    popMatrix();
}
```

1.3. Actividad 6.3: Interfaz unificada con joystick y servomotor

Enunciado: Desarrolle una aplicación en Processing que integre la lectura física de un módulo joystick analógico con el control de un servomotor. El eje que presente mayor desviación respecto al centro de reposo debe ser el eje dominante y controlar el servo: el eje X mapeará ángulos de 0° a 90° y el eje Y de 90° a 180°. La interfaz gráfica debe mostrar los valores ADC de cada eje, el estado del pulsador y una representación visual dinámica del brazo del servo.

El joystick KY-023 expone tres señales: dos análogas de posición (VRX y VRY) y un pulsador digital (SW). Las dos primeras se conectaron a las entradas analógicas A0 y A1 respectivamente, y el pulsador al pin digital 2. Para el pulsador se activó la resistencia *pullup* interna, de modo que su valor en reposo es HIGH y pasa a LOW al presionarlo. El servo se conectó al pin 3, que Firmata gestiona como salida de posicionamiento automático [1].

La declaración de pines y la configuración del hardware en `setup()` se muestra en el Código 6:

Código 6: Declaración de pines y configuración en `setup()` del sketch unificado.

```
int pinVRX = 0; // Entrada analogica A0 (Controla 0 a 90 grados)
int pinVRY = 1; // Entrada analogica A1 (Controla 90 a 180 grados)
int pinSW = 2;  // Entrada digital Pin 2 (Pulsador)
int servoPin = 3; // Pin de control del servo motor

arduino = new Arduino(this, Arduino.list()[2], 57600);
arduino.pinMode(pinSW, Arduino.INPUT);
arduino.digitalWrite(pinSW, Arduino.HIGH); // Activar pullup interno
arduino.pinMode(servoPin, Arduino.SERVO);
arduino.servoWrite(servoPin, 90);        // Posicion inicial segura
```

Dentro de `draw()`, primero se leen los tres valores del hardware. Luego se aplica la *lógica de eje dominante*: se compara cuál de los dos ejes se aleja más del valor central de reposo (512 en un ADC de 10 bits) y se usa ese eje para calcular el ángulo. Si el eje X domina, el valor ADC se mapea al rango $[0^\circ, 90^\circ]$; si domina el eje Y, al rango $[90^\circ, 180^\circ]$ (Código 7). Esto impide que ambos ejes interfieran entre sí y que el servo reciba órdenes contradictorias.

Código 7: Lectura del joystick y lógica de eje dominante en `draw()`.

```
valorX = arduino.analogRead(pinVRX);
valorY = arduino.analogRead(pinVRY);
estadoSW = arduino.digitalRead(pinSW);

if (abs(valorX - 512) > abs(valorY - 512)) {
    // Eje X domina: rango 0 a 90 grados
    anguloServo = (int) map(valorX, 0, 1023, 0, 90);
} else {
    // Eje Y domina: rango 90 a 180 grados
    anguloServo = (int) map(valorY, 0, 1023, 90, 180);
}
arduino.servoWrite(servoPin, anguloServo);
```

La interfaz gráfica muestra en pantalla los valores ADC de cada eje, el estado del pulsador y un panel que contiene la representación visual del brazo del servo. La función `dibujarServoGrafico()` centra la transformación en el punto de pivote, rota la línea del brazo y dibuja el eje central como una elipse (Código 8). El ajuste de -90° en la rotación se debe a que en el sistema de coordenadas de Processing 0° apunta hacia la derecha, mientras que visualmente se desea que 90° apunte hacia arriba.

Código 8: Función auxiliar para dibujar el brazo del servo en pantalla.

```
void dibujarServoGrafico(int x, int y, int angulo) {
    pushMatrix();
    translate(x, y);
    rotate(radians(angulo - 90)); // 90 deg apunta arriba en pantalla
    stroke(0, 255, 100);
    strokeWeight(6);
    line(0, 0, 0, -50); // Brazo mecanico
    fill(255);
    noStroke();
    ellipse(0, 0, 15, 15); // Eje central circular
    popMatrix();
}
```

La llamada a `delay(20)` al final de `draw()` introduce una pausa breve para estabilizar la tasa de actualización del bus Firmata y evitar que lecturas demasiado rápidas saturen la comunicación serial [1].

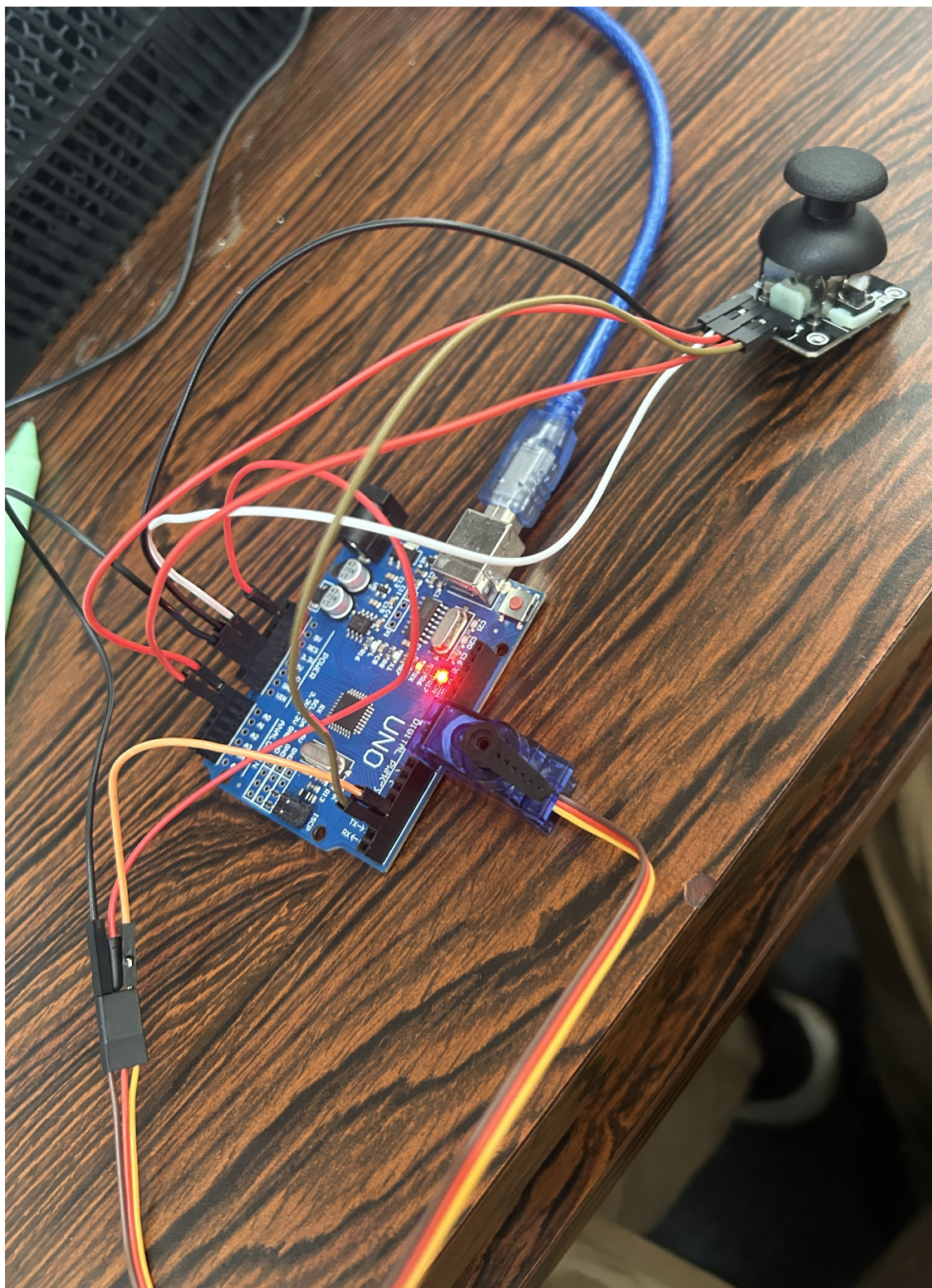


Figura 6: IMG

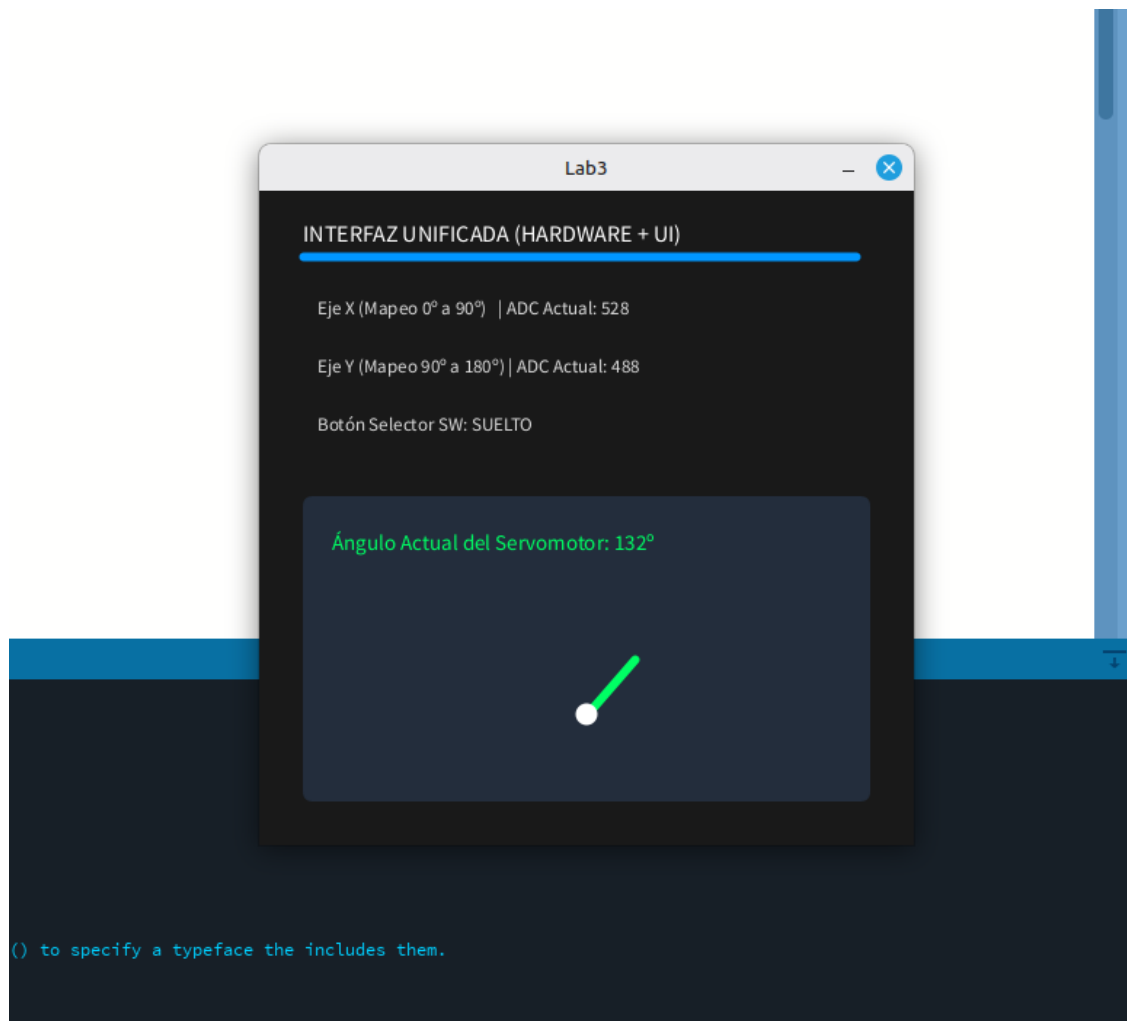


Figura 7: IMG

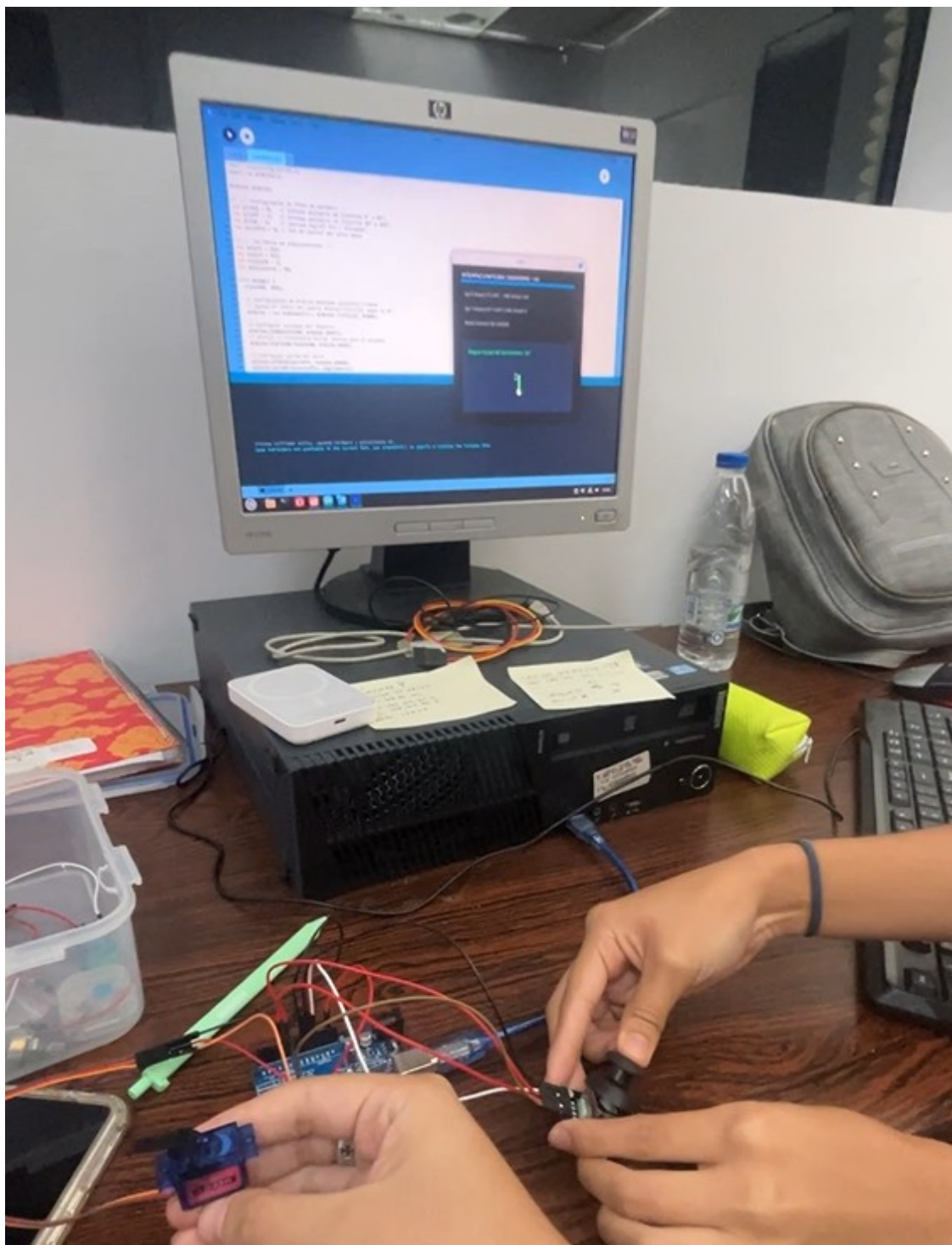


Figura 8: IMG

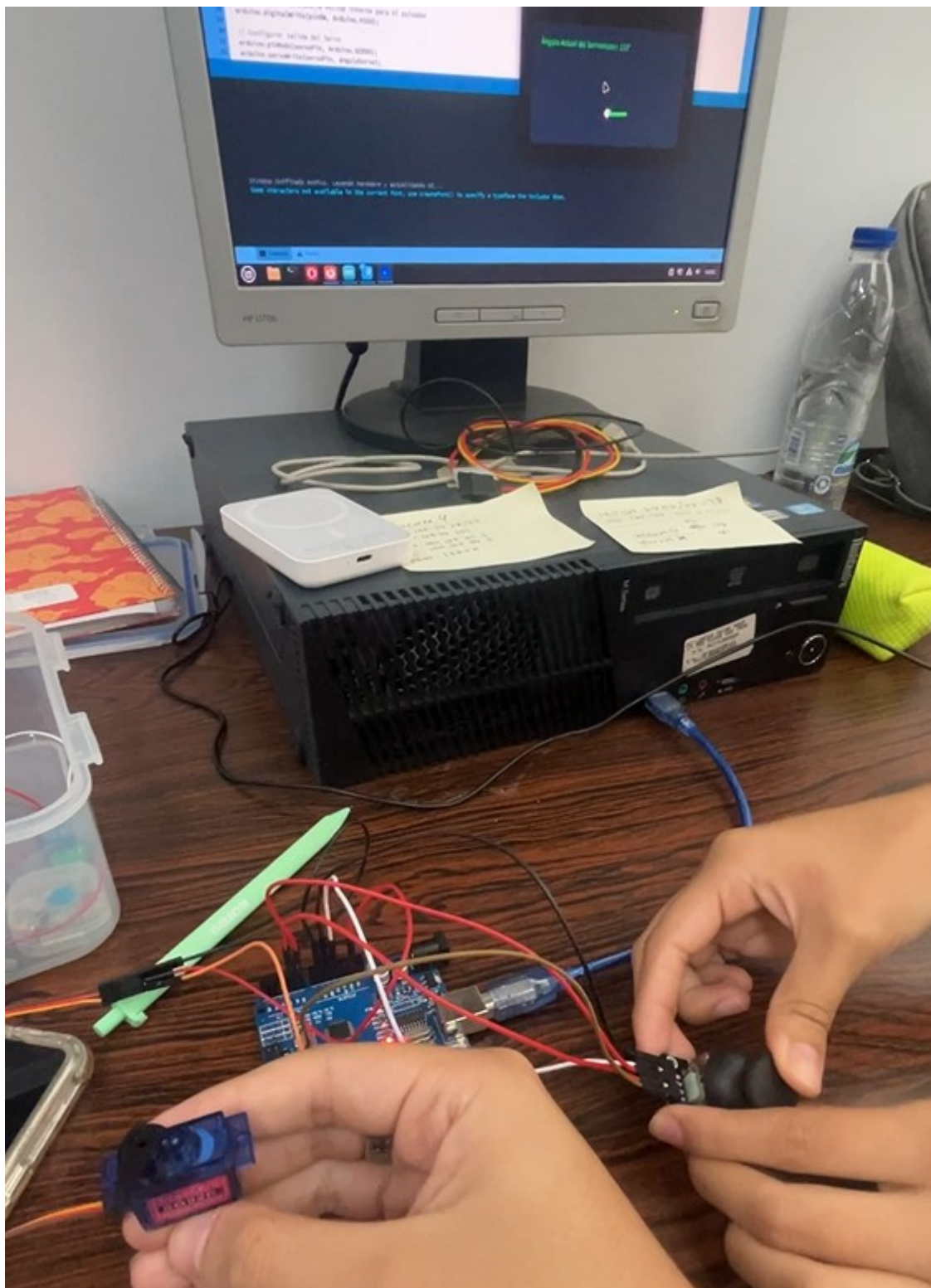


Figura 9: IMG

2. Conclusiones

En este laboratorio se exploró cómo extender el control de hardware desde una interfaz local hacia una arquitectura cliente-servidor distribuida mediante sockets TCP.

En la actividad 6.1 se comprobó que la lógica de eje dominante resuelve de forma limpia el problema de la interferencia entre los dos ejes del joystick. Al comparar la desviación absoluta respecto al centro ($|x - 512|$ vs. $|y - 512|$), solo el eje con mayor movimiento tiene efecto sobre el servo, lo que hace el control intuitivo y predecible sin necesidad de zonas muertas adicionales.

En la actividad 6.2 se confirmó que TCP, al ser orientado a conexión, garantiza la entrega ordenada de los datos, pero no define bordes de mensaje. El uso del carácter '`\n`' como delimitador junto con `readStringUntil()` es la solución más simple para reconstruir mensajes completos en el lado receptor. La validación del rango antes de escribir al servo resultó indispensable para la integridad del hardware.

En la actividad 6.3 se pudo apreciar la ventaja del callback `controlEvent()` frente a la lectura polinómica en `draw()`: solo se genera tráfico de red cuando el usuario realmente interactúa con el knob del servo, en lugar de enviar datos a cada fotograma. La diferenciación entre el puerto externo (8080) y el puerto interno del servidor (12345) ilustra un caso típico de despliegue IoT con reenvío de puertos en un router NAT.

Referencias

- [1] F. Mellis, H. Barragan y otros, “Firmata Protocol Documentation,” *Firmata GitHub Repository*, 2006–2024. [En línea]. Disponible: <https://github.com/firmata/protocol>. [Consulta: jun. 2026]
- [2] The Processing Foundation, “processing/net — Network Library,” *Processing Reference*, 2001–2024. [En línea]. Disponible: <https://processing.org/reference/libraries/net/>. [Consulta: jun. 2026]
- [3] A. Cockpit, “controlP5 — A GUI Library for Processing,” 2012–2023. [En línea]. Disponible: <https://www.sojamo.de/libraries/controlP5/>. [Consulta: jun. 2026]
- [4] Arduino, “Arduino Language Reference,” *Arduino Documentation*, Arduino LLC, 2024. [En línea]. Disponible: <https://docs.arduino.cc/language-reference/>. [Consulta: jun. 2026]
- [5] A. Russoniello, “Laboratorio: Comunicación TCP y Control Remoto con Processing,” guía de práctica de laboratorio, Laboratorio General (Internet de las Cosas), Escuela de Computación, Facultad de Ciencias, Universidad Central de Venezuela, Caracas, 2026.